

Function Calling 的本质，是让模型在需要外部能力时，先“提出调用某个函数/工具的请求”，由你的程序真正执行函数，再把执行结果交还给模型，让模型基于真实结果继续回答。

1. Function Calling 解决什么问题？

传统大模型有几个天然限制：

1. 不知道训练后发生的新信息
比如今天巴黎天气、某个订单状态、某只股票最新价格。
2. 不能直接访问你的私有系统
比如 CRM、订单系统、数据库、公司知识库、内部 API。
3. 不能真正执行动作
比如发邮件、退款、创建日程、提交工单、运行代码。
4. 生成内容不一定结构化
你让模型输出 JSON，它可能少字段、多字段、格式错误。

Function Calling 就是为了解决这些问题：

把模型变成“决策者 / 参数生成器”，把外部程序变成“执行者”。

例如用户问：

“帮我查一下巴黎今天的天气。”

模型不会凭空编一个天气，而是返回类似：

```
{
  "name": "get_weather",
  "arguments": {
    "location": "Paris",
    "units": "celsius"
  }
}
```

然后你的应用程序真的去调用天气 API，得到：

```
{
  "temperature": 25,
  "unit": "C"
}
```

再把结果发回模型，模型最终回答：

“巴黎今天大约 25°C。”

2. 核心概念：Tool、Tool Call、Tool Call Output

这篇文档首先建立了三个关键概念。

2.1 Tool：你提供给模型的能力

Tool 是你告诉模型“你可以用的功能”。

它可以是：

1. 查询天气；
2. 查询用户账户；
3. 查询订单；
4. 申请退款；
5. 搜索文档；
6. 执行代码；
7. 调用 MCP server；
8. 调用你自己公司的内部系统。

在 OpenAI API 里，最常见的是 **function tool**，也就是用 JSON Schema 描述一个函数。

例如：

```
{
  "type": "function",
  "name": "get_weather",
  "description": "Retrieves current weather for the given location.",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City and country e.g. Bogotá, Colombia"
      },
      "units": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"]
      }
    }
  }
}
```

```
    },
    "required": ["location", "units"],
    "additionalProperties": false
  },
  "strict": true
}
```

这里你并不是把函数代码给模型，而是把 **函数说明书** 给模型。

模型知道：

有一个叫 `get_weather` 的工具，
它需要 `location` 和 `units` 两个参数，
`units` 只能是 `celsius` 或 `fahrenheit` 。

2.2 Tool Call：模型提出的调用请求

Tool Call 是模型说：

“我认为现在需要调用这个工具，并且参数应该是这些。”

比如用户问：

“What's the weather in Paris?”

模型可能返回：

```
{
  "type": "function_call",
  "name": "get_weather",
  "arguments": "{\"location\":\"Paris\",\"units\":\"celsius\"}"
}
```

注意：

这一步只是 **模型请求调用工具**，并没有真的执行。

2.3 Tool Call Output：你的程序执行后的结果

Tool Call Output 是你的应用程序真正调用函数后，把结果返回给模型。

比如你的程序调用天气 API 后得到：

```
{  
  "temperature": 14,  
  "unit": "C"  
}
```

然后你把这个结果和前面的对话一起再发回模型。

模型最终生成自然语言回答：

“It’s currently 14°C in Paris.”

文档第 4 页的图非常清晰地展示了这个流程：

开发者先把工具定义和用户问题发给模型；模型返回 `get_weather("paris")` 的工具调用；开发者执行函数，得到 `{"temperature": 14}`；再把结果发回模型；模型最终回答“巴黎现在 14°C”。

3. Function Calling 的完整流程

文档把 Tool Calling 描述成一个多步骤流程：

Step 1: 开发者向模型发送请求，并附上可用工具

你告诉模型：

你现在可以使用这些工具：`get_weather`、`search_docs`、`refund_order` 等。

同时发送用户问题。

Step 2: 模型决定是否调用工具

模型会判断：

1. 不需要工具，直接回答；
2. 需要调用一个工具；
3. 需要调用多个工具；
4. 需要继续调用工具链。

例如：

用户问：“什么是 Transformer？”

模型可以直接回答。

用户问：“查一下我订单 123 的状态。”

模型需要调用订单查询工具。

Step 3: 你的应用程序执行工具

模型只负责生成：

```
{
  "name": "get_order_status",
  "arguments": {
    "order_id": "123"
  }
}
```

你的应用程序负责：

```
result = get_order_status(order_id="123")
```

这里真正查数据库、调用后端 API、验证权限、处理异常，都是你的代码做。

Step 4: 把工具结果返回给模型

比如：

```
{
  "order_id": "123",
  "status": "shipped",
  "estimated_delivery": "2026-05-28"
}
```

Step 5: 模型生成最终回答，或者继续调用工具

模型可能回答：

“订单 123 已发货，预计 5 月 28 日送达。”

也可能继续调用另一个工具，比如：

查物流详情。

所以 Function Calling 不是单轮动作，而是可以构成多轮 Agent 流程。

4. Function 与 Tool 的区别

文档中特别区分了 **function** 和 **tool**。

Function 是一种特定类型的 Tool

Function 是用 JSON Schema 描述的工具，适合结构化输入输出。

例如：

```
{
  "type": "function",
  "name": "get_customer_profile",
  "parameters": {
    "type": "object",
    "properties": {
      "customer_id": {
        "type": "string"
      }
    },
    "required": ["customer_id"]
  }
}
```

这类工具很适合：

1. 查数据库；
2. 调 API；
3. 创建订单；
4. 查询日程；
5. 搜索文档；
6. 执行业务系统动作。

Tool 是更大的概念

Tool 包括：

1. Function tools

JSON Schema 驱动的函数工具。

2. Custom tools

输入可以是自由文本，不一定是 JSON。

3. Built-in tools

OpenAI 平台内置工具，例如网页搜索、代码执行、MCP 工具等。

所以：

Function 是 Tool 的一种；

Tool 是模型可调用外部能力的统称。

5. 如何定义一个 Function?

文档给出的 function definition 通常包含 5 个字段。

字段	含义
type	固定为 "function"
name	函数名，例如 <code>get_weather</code>
description	说明什么时候、怎么使用这个函数
parameters	用 JSON Schema 定义输入参数
strict	是否启用严格模式

其中最重要的是三个部分：

5.1 name：函数名要清楚

不要写：

```
"name": "handle"
```

要写：

```
"name": "get_current_weather"
```

因为模型是根据函数名和描述来判断是否调用工具的。

5.2 description：告诉模型什么时候用

差的描述：

```
"description": "Get data."
```

好的描述：

```
"description": "Retrieve the current weather for a city and country. Use this when the user asks about today's weather, temperature, or current conditions."
```

描述越清楚，模型越不容易误用。

5.3 parameters：用 JSON Schema 约束参数

例如：

```
"parameters": {
  "type": "object",
  "properties": {
    "location": {
      "type": "string",
      "description": "City and country, e.g. Bogotá, Colombia"
    },
    "units": {
      "type": "string",
      "enum": ["celsius", "fahrenheit"]
    }
  },
  "required": ["location", "units"],
  "additionalProperties": false
}
```

这里有几个关键点：

1. `type` 定义字段类型；
2. `description` 解释字段含义；
3. `enum` 限制可选值；
4. `required` 指定必填字段；
5. `additionalProperties: false` 禁止模型生成额外字段。

这就像给模型一张表格，让它只能按表格填。

6. Strict Mode：为什么建议打开？

文档建议：**始终开启 strict mode**。

```
"strict": true
```

Strict Mode 的作用是：

让模型生成的函数调用严格符合你定义的 JSON Schema，而不是“尽量符合”。

不开 strict 的时候，模型可能会生成：

```
{
  "location": "Paris",
  "unit": "C"
}
```

但你的 schema 需要的是：

```
{
  "location": "Paris",
  "units": "celsius"
}
```

字段名不一致，就会导致程序解析失败。

开启 strict 后，模型更可靠地遵循 schema。

不过 Strict Mode 有要求：

1. 每个 object 都要设置 `additionalProperties: false` ；
2. `properties` 中的字段都要放入 `required` ；
3. 如果某字段可选，可以用 `null` 类型表示；
4. 有些 JSON Schema 特性不被支持；
5. schema 第一次使用时会经过额外处理并缓存，可能增加首次延迟。

通俗理解：

strict mode 就是“填表必须严格按表头来，不能自己加列，也不能漏列”。

7. Function Calling 的最佳实践

这是文档中非常重要的一部分，适合你未来做 Agent / AI 应用架构时重点掌握。

7.1 函数名、参数名、描述都要清楚

模型不是靠魔法理解工具，而是靠你给它的说明。

好的工具定义应该让一个实习生也能正确使用。

文档提到一个很有用的标准：

****Intern Test：如果一个实习生只看你给模型的函数说明，能不能正确调用这个函数？****

如果不能，说明你的函数描述不够清楚。

例如，不要只写：

```
"description": "Search docs."
```

而应该写：

```
"description": "Search the internal research report database. Use this when  
the user asks about historical reports, analyst views, industry research, or  
company coverage."
```

7.2 用 enum 和结构化字段减少错误状态

文档给了一个典型反例：

```
toggle_light(on: bool, off: bool)
```

这个设计很差，因为可能出现：

```
{  
  "on": true,
```

```
"off": true
}
```

这就是非法状态。

更好的设计是：

```
{
  "action": "turn_on"
}
```

并用 enum 限制：

```
"enum": ["turn_on", "turn_off"]
```

原则是：

| 让非法状态无法被表达。

这和软件工程里的类型设计思想一致。

7.3 不要让模型填你已经知道的参数

如果你的程序上下文里已经有 `order_id`，不要让模型再生成一次 `order_id`。

差的设计：

```
{
  "name": "submit_refund",
  "parameters": {
    "order_id": "string"
  }
}
```

如果当前页面本来就是订单 123 的详情页，你的代码已经知道 `order_id`，那更好的设计是：

```
{
  "name": "submit_refund",
  "parameters": {}
}
```

然后由你的代码把当前页面里的 `order_id` 传进去。

原因是：

模型每多填一个参数，就多一个出错机会。

已知信息应该由代码传递，不要让模型重复推断。

7.4 把总是连续调用的函数合并

如果你总是这样调用：

```
query_location()  
mark_location()
```

那就不要设计成两个工具。

可以合并成：

```
query_and_mark_location()
```

原则是：

不要让模型负责机械性的流程编排；能写进代码的就写进代码。

模型适合做判断，不适合做稳定的低层流程控制。

7.5 一开始不要暴露太多工具

文档建议：初始可用函数数量尽量少，软建议是少于 20 个。

原因是工具太多时，模型会更容易：

1. 选错工具；
2. 混淆工具用途；
3. 生成错误参数；
4. 消耗大量 token；
5. 降低响应速度。

这点对 Agent 系统非常关键。

你可以把工具想象成给人一本菜单：

- 菜单只有 8 个选项时，人很容易选；
- 菜单有 200 个选项时，人会犹豫，也更容易选错。

8. Tool Search：工具太多怎么办？

如果你的应用有很多工具，比如几十个、上百个，文档建议使用 **Tool Search**。

Tool Search 的思路是：

- 一开始不要把所有工具都塞给模型。
- 先让模型搜索需要哪些工具，再把相关工具加载进上下文。

这类似 RAG：

- RAG 是先搜索相关文档，再把文档放进上下文；
- Tool Search 是先搜索相关工具，再把工具定义放进上下文。

文档中提到，Tool Search 适合大型工具生态，并且只有较新的模型支持。

它解决的是两个问题：

1. **准确性问题**：工具少，模型更容易选对；
2. **成本问题**：工具定义也占 token，不要把所有 schema 都塞进去。

通俗理解：

- 不要把整个公司所有 API 文档都给模型。
- 先让模型判断：“我现在可能需要 CRM 工具。”
- 然后只加载 CRM 相关工具。

9. Namespace：给工具分组

文档提到可以用 **namespace** 把相关工具按领域分组。

例如：

```
{  
  "type": "namespace",  
  "name": "crm",  
  "description": "CRM tools for customer lookup and order management.",  
}
```

```
"tools": [  
  {  
    "type": "function",  
    "name": "get_customer_profile"  
  },  
  {  
    "type": "function",  
    "name": "list_open_orders"  
  }  
]
```

Namespace 的作用是帮助模型区分不同系统。

比如你可能有：

1. `crm.search_customer`
2. `billing.search_invoice`
3. `support.search_ticket`
4. `research.search_report`

它们都叫 `search`，但搜索对象完全不同。

Namespace 可以减少混淆。

10. Token Usage：工具定义也会消耗上下文

文档特别提醒：函数定义会被注入到系统消息中，因此：

1. 会占用 context window；
2. 会作为 input token 计费；
3. schema 越长，成本越高；
4. 工具越多，输入越重；
5. 工具描述太详细也会增加 token。

所以如果遇到上下文限制，可以：

1. 减少初始工具数量；
2. 缩短函数描述；
3. 使用 Tool Search 延迟加载；
4. 对大量函数调用场景做 fine-tuning；
5. 把复杂逻辑放到代码里，而不是放到工具描述里。

这对生产系统很重要。

Function Calling 不是“工具越多越强”，而是要做工具面设计。

11. 如何处理模型返回的 Function Call?

文档强调：模型一次响应可能返回 **0 个、1 个或多个 function calls** 。

所以你的代码不要默认只有一个工具调用。

更稳妥的逻辑是：

```
for tool_call in response.output:
    if tool_call.type == "function_call":
        result = call_function(tool_call.name, tool_call.arguments)
        append_tool_result(tool_call.call_id, result)
```

这里有个关键字段： `call_id` 。

它的作用是：

把某个工具调用和对应工具结果绑定起来。

因为模型可能一次调用多个函数，比如：

1. `get_weather("Paris")`
2. `get_weather("London")`
3. `get_weather("New York")`

你返回结果时，必须说明每个结果对应哪个 call。

12. Tool Call Output 应该怎么格式化?

文档说，工具结果通常可以是字符串，字符串里可以放：

1. JSON；
2. 普通文本；
3. 错误码；
4. 成功 / 失败信息。

例如：

```
{  
  "status": "success",  
  "temperature": 25,  
  "unit": "C"  
}
```

如果函数没有返回值，比如 `send_email`，也应该返回一个结果：

```
success
```

或者：

```
{  
  "status": "success",  
  "message": "Email sent."  
}
```

不要什么都不返回。

因为模型需要知道动作是否成功。

13. Tool Choice：如何控制模型是否调用工具？

默认情况下，模型会自动决定是否调用工具。

但你可以通过 `tool_choice` 控制行为。

文档列了几种模式。

13.1 Auto：默认模式

```
"tool_choice": "auto"
```

模型可以：

1. 不调用工具；
2. 调用一个工具；
3. 调用多个工具。

适合大多数场景。

13.2 Required：必须调用工具

```
"tool_choice": "required"
```

模型必须调用至少一个工具。

适合你明确要求模型走工具流程的场景，比如：

用户问的所有问题都必须先查内部知识库，不能直接回答。

13.3 Forced Function：强制调用某个函数

```
"tool_choice": {  
  "type": "function",  
  "name": "get_weather"  
}
```

这表示模型必须调用 `get_weather` 。

适合流程很确定的场景。

13.4 Allowed Tools：限制只能调用某些工具

```
"tool_choice": {  
  "type": "allowed_tools",  
  "mode": "auto",  
  "tools": [  
    { "type": "function", "name": "get_weather" },  
    { "type": "function", "name": "search_docs" }  
  ]  
}
```

这表示虽然你传了很多工具，但当前轮只允许模型从某个子集中选择。

这个设计适合做权限控制、流程控制、阶段控制。

比如一个客服 Agent：

1. 问题识别阶段：只允许调用分类工具；
2. 查询阶段：允许查订单、查物流；
3. 操作阶段：允许退款、改地址，但需要额外确认。

14. Parallel Function Calling：并行工具调用

模型可能在一轮里调用多个函数。

例如用户问：

“帮我比较巴黎、伦敦、纽约今天的天气。”

模型可以一次返回三个工具调用：

```
get_weather("Paris")
get_weather("London")
get_weather("New York")
```

你的程序可以并行执行，提高效率。

如果不希望模型一轮调用多个工具，可以设置：

```
"parallel_tool_calls": false
```

这样模型每轮最多调用一个工具。

文档也提醒：使用某些 built-in tools 时，并行函数调用不可用；部分 fine-tuned model 在多工具调用时 strict mode 可能被禁用。

通俗理解：

并行调用适合“多个独立查询”；

串行调用适合“后一步依赖前一步结果”。

15. Streaming：流式工具调用

文档还讲了 Streaming 场景。

普通流式输出是模型一点点吐文字。

Function Calling 的流式输出则是模型一点点生成函数参数。

比如模型正在生成：

```
{  
  "location": "Paris",  
  "units": "celsius"  
}
```

你可能会收到多个 delta 片段，需要自己拼起来。

事件大致包括：

1. `response.output_item.added`
表示出现了一个新的 function call item。
2. `response.function_call_arguments.delta`
表示函数参数的一部分增量。
3. `response.function_call_arguments.done`
表示参数生成完成，可以执行函数。

这适合需要展示“模型正在准备调用哪个工具”的产品体验。

例如前端可以显示：

正在查询天气.....
正在调用 `get_weather`
参数：Paris, celsius.....

16. Custom Tools：不一定非要 JSON Schema

除了 function tools，文档还介绍了 **custom tools** 。

Function tool 的输入是 JSON Schema 定义的结构化参数。
Custom tool 的输入可以是自由文本。

适合什么场景？

比如代码执行工具：

```
print("hello world")
```

这时候没必要包成：

```
{  
  "code": "print(\"hello world\")"  
}
```

可以直接让模型把代码作为字符串传给工具。

另一个例子是数学表达式工具：

```
4 + 4
```

模型直接把表达式传给工具即可。

所以：

Function tools 适合业务 API；

Custom tools 适合代码、公式、DSL、查询语言、自由文本处理。

17. CFG / Grammar：约束 Custom Tool 的自由文本格式

Custom tool 虽然可以接受自由文本，但自由文本也可能失控。

所以文档介绍了 **Context-Free Grammar, CFG**，用于约束模型传给 custom tool 的文本格式。

OpenAI 支持两类 grammar：

1. **Lark**
2. **Regex**

比如你有一个数学表达式工具，只想允许：

```
4 + 4
```

不想让模型输出一大段解释，那么可以用 grammar 限制输出必须是某种表达式格式。

17.1 Lark Grammar

Lark 是一种语法定义方式，可以定义：

1. 数字；
2. 运算符；

3. 表达式；
4. 括号；
5. 语句结构。

例如简单表达式：

```
start: expr
NUMBER: /[0-9]+/
PLUS: "+"
MINUS: "-"
expr: term (("+"|" -") term)*
term: NUMBER
```

这个 grammar 表示：

| 表达式由数字和加减号组成。

文档也提醒，Lark grammar 不要写得太复杂，否则 API 可能报错，模型也更容易 out-of-distribution。

17.2 Regex Grammar

Regex grammar 用正则表达式约束模型输出。

例如约束时间格式、日期格式、简单代码片段等。

文档提醒：

1. pattern 必须写在一行；
2. 不要用 Python re 的语法想当然处理；
3. 使用的是 Rust regex crate 语法；
4. 不支持 lookahead；
5. 不支持 lazy modifiers，如 `*?`、`+?`、`??`。

18. Grammar 的关键注意事项

文档后半部分很工程化，重点讲：**不要把 grammar 写得太复杂，也不要误用 parser rule 去控制 lexer 行为。**

这部分可以简化理解为以下几点。

18.1 Lexer 先于 Parser 运行

Lark 里有两个层次：

1. **Terminal**：词法 token，通常大写；
2. **Rule**：语法规则，通常小写。

问题在于：

Lexer 会先把文本切成 token，然后 parser 才应用规则。

所以如果你试图用 parser rule 去控制自由文本怎么切分，往往会失败。

18.2 处理自由文本时，尽量用一个完整 terminal

文档给了一个重要建议：

如果你想匹配“某些锚点之间的任意自然语言”，最好用一个完整的 regex terminal，而不是拆成多个 rule。

因为 lexer 是贪婪匹配，拆得太碎可能导致边界失控。

通俗说：

你想让模型输出一句固定结构的自然语言，不要把“主语、动词、宾语之间的任意文本”拆得太复杂。
复杂拆分会让 lexer 抢先乱切。

18.3 避免无边界的正则

不要滥用：

```
/.+/
```

或者到处用：

```
.*
```

更好的做法是：

```
/[^\n]{0,100}\./
```

也就是给范围和长度加边界。

原因是：

1. 减少模型输出过长；
2. 减少 grammar 复杂度；
3. 降低 out-of-distribution；
4. 提高工具调用稳定性。

19. Function Calling 与 Agent 的关系

这份文档虽然讲的是 Function Calling，但它实际上是 Agent 系统的底层机制之一。

一个简单 Agent 循环可以表示为：

```
User Input
  ↓
Model Reasoning
  ↓
Tool Call
  ↓
Tool Execution
  ↓
Observation / Tool Result
  ↓
Model Continues
  ↓
Final Answer or More Tool Calls
```

这和 ReAct 模式非常接近：

```
Reason → Act → Observe → Reason → Act → Observe → Answer
```

Function Calling 里的：

- `function_call` 对应 **Act**
- `function_call_output` 对应 **Observe**
- 模型下一轮回答对应 **Reason / Answer**

所以可以把 Function Calling 理解为：

OpenAI API 层面对 ReAct / Agent 工具调用的标准化实现方式。==

20. 对 AI 应用开发的关键启发

结合这份文档，最重要的不是记 API 参数，而是理解背后的系统设计原则。

20.1 模型负责“判断与生成参数”，代码负责“执行与校验”

不要让模型做所有事。

模型适合：

1. 理解用户意图；
2. 判断该用哪个工具；
3. 生成结构化参数；
4. 解释工具返回结果；
5. 在信息不足时追问。

代码适合：

1. 权限校验；
2. 数据库查询；
3. API 调用；
4. 状态变更；
5. 参数二次校验；
6. 错误处理；
7. 日志记录；
8. 幂等控制。

这是一条非常重要的边界。

20.2 工具设计决定 Agent 稳定性

很多 Agent 不稳定，不是因为模型差，而是因为工具设计差。

差的工具设计包括：

1. 工具太多；
2. 命名模糊；
3. 参数过多；
4. 参数描述不清；
5. 没有 enum；
6. 没有 strict mode；
7. 把底层细节暴露给模型；
8. 让模型承担本该由代码完成的流程编排。

好的工具设计应该是：

1. 名字明确；
2. 参数少而清楚；
3. schema 严格；
4. 函数粒度适中；
5. 工具数量可控；
6. 错误结果可解释；
7. 能通过 Intern Test。

20.3 Function Calling 不是让模型“真的执行函数”

这是很多初学者容易误解的地方。

模型不会真的执行：

```
get_weather("Paris")
```

它只是返回一个调用请求。

真正执行发生在你的应用程序里。

所以 Function Calling 的架构是：

模型：我想调用 `get_weather`，参数是 `Paris`

应用：收到，我来真的调用天气 API

天气 API：返回 `14°C`

应用：把结果交给模型

模型：组织成自然语言回答用户

20.4 工具调用需要安全边界

因为工具可能执行真实动作，比如：

1. 发邮件；
2. 删除数据；
3. 创建订单；
4. 执行退款；
5. 修改账户；
6. 调用交易系统。

所以必须设计安全机制：

1. 敏感操作前让用户确认；
2. 服务端校验权限；
3. 不完全信任模型生成的参数；
4. 设置幂等 key；
5. 记录审计日志；
6. 对高风险工具设置 allowed tools；
7. 区分 read-only 工具和 write 工具；
8. 对外部输入做 sanitization。

尤其在金融场景，不能让模型直接决定“下单”“调仓”“付款”“退款”等动作，必须有人工确认或严格规则。

21. 用金融场景举例：研报问答 Agent

假设你要做一个“研报问答 Agent”，可以这样设计 function tools。

21.1 搜索研报

```
{
  "type": "function",
  "name": "search_research_reports",
  "description": "Search internal research reports by company, industry, analyst, keyword, or date range.",
  "parameters": {
    "type": "object",
```

```
"properties": {
  "query": {
    "type": "string",
    "description": "Search query, e.g. company name, industry, or topic."
  },
  "date_range": {
    "type": ["string", "null"],
    "description": "Optional date range, e.g. '2024-01-01 to 2024-12-31'."
  },
  "top_k": {
    "type": "integer",
    "description": "Number of results to return."
  }
},
"required": ["query", "date_range", "top_k"],
"additionalProperties": false
},
"strict": true
}
```

21.2 获取研报正文片段

```
{
  "type": "function",
  "name": "get_report_chunks",
  "description": "Retrieve relevant chunks from a specific research report by report ID.",
  "parameters": {
    "type": "object",
    "properties": {
      "report_id": {
        "type": "string"
      },
      "chunk_ids": {
        "type": "array",
        "items": {
          "type": "string"
        }
      }
    }
  },
  "required": ["report_id", "chunk_ids"],
  "additionalProperties": false
},
```

```
"strict": true
}
```

21.3 生成引用溯源

```
{
  "type": "function",
  "name": "format_citations",
  "description": "Format report citations using report title, analyst,
publication date, and page number.",
  "parameters": {
    "type": "object",
    "properties": {
      "source_ids": {
        "type": "array",
        "items": {
          "type": "string"
        }
      }
    }
  },
  "required": ["source_ids"],
  "additionalProperties": false
},
"strict": true
}
```

这个场景里，模型不应该凭空回答，而应该：

```
用户问题
↓
调用 search_research_reports
↓
调用 get_report_chunks
↓
基于片段回答
↓
调用 format_citations 或直接生成引用
```

这样才能保证金融问答的可追溯性。

22. 用你的 Market Insight Agent 类比理解

你之前做的 Market Insight Agent，其实也可以用 Function Calling 的视角重构。

原来的 Agent 流程大致是：

```
异常候选数据
↓
LLM 判断是否需要搜索新闻
↓
search_market_news
↓
生成异动解释卡片
```

对应 Function Calling：

```
{
  "type": "function",
  "name": "search_market_news",
  "description": "Search recent market news related to a given asset, macro event, or anomaly.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string"
      },
      "recency_days": {
        "type": "integer"
      }
    },
    "required": ["query", "recency_days"],
    "additionalProperties": false
  },
  "strict": true
}
```

模型看到：

```
{
  "asset": "10Y US Treasury Yield",
  "weekly_change": "+25bp",
  "historical_percentile": 95,
```

```
"severity_score": 0.91
}
```

它判断：

这个异动可能需要外部新闻解释，于是调用 `search_market_news` 。

工具返回新闻线索后，模型生成：

1. 事实；
2. 幅度；
3. 影响；
4. 可能驱动；
5. 置信度。

这正是 Function Calling 的典型用法。

23. 和 RAG 的区别与结合

Function Calling 和 RAG 经常一起用，但不是一回事。

RAG 解决的是“查知识”

RAG 通常流程是：

```
用户问题
↓
embedding / keyword search
↓
取回相关文档片段
↓
模型基于片段回答
```

它主要解决：

1. 知识更新；
2. 私有知识库；
3. 引用溯源；
4. 减少幻觉。

Function Calling 解决的是“调用能力”

Function Calling 可以调用：

1. 搜索工具；
2. 数据库；
3. 计算器；
4. 日程系统；
5. 邮件系统；
6. 订单系统；
7. 代码执行器；
8. RAG 检索器。

所以 RAG 可以作为一个 Function Tool。

例如：

```
{  
  "name": "search_knowledge_base",  
  "description": "Search the company knowledge base for relevant documents."  
}
```

通俗说：

RAG 是一种工具能力；
Function Calling 是让模型选择和调用这种能力的机制。

24. 常见错误理解

误解 1：Function Calling 会自动执行函数

不会。

模型只是生成调用请求。
你必须写代码执行函数。

误解 2：给模型越多工具越好

不是。

工具越多，选择越难，token 成本越高，误调用概率越大。

误解 3：有了 Function Calling 就不会幻觉

不一定。

Function Calling 可以降低幻觉，但不能消除幻觉。

如果工具返回结果不完整、模型解释过度，仍然会出错。

误解 4：工具参数可以随便写

不行。

参数设计是核心工程问题。

要用 JSON Schema、enum、required、strict mode 来控制。

误解 5：模型可以替代后端权限系统

绝对不行。

模型生成的工具调用必须经过后端权限校验。

尤其是涉及写操作、交易、付款、删除、发邮件等动作。

25. 真正掌握的 10 个要点

1. Function Calling 是模型与外部系统交互的标准机制。
 2. 模型只生成 tool call，不真正执行函数。
 3. 真实函数执行发生在你的应用程序侧。
 4. 工具定义本质上是给模型看的“API 使用说明书”。
 5. JSON Schema 决定模型能生成什么参数。
 6. Strict Mode 可以显著提高结构化调用的可靠性。
 7. 工具数量不要太多，初始工具面越小越稳定。
 8. Tool Choice 可以控制模型是否调用工具、调用哪些工具。
 9. Custom Tools 适合自由文本、代码、DSL、grammar 约束场景。
 10. Function Calling 是 Agent 系统中 Act / Observe 循环的基础设施。
-

26. 用一句话区分几个相关概念

概念	一句话解释
Function Calling	模型请求调用你定义的函数
Tool Calling	更广义的工具调用，包括 function/custom/built-in tools
Tool	模型可用的外部能力
Function Tool	用 JSON Schema 描述的结构化工具
Custom Tool	输入可以是自由文本的工具
Tool Call	模型生成的调用请求
Tool Call Output	应用程序执行工具后返回给模型的结果
Strict Mode	强制函数调用符合 schema
Tool Choice	控制模型是否/如何调用工具
Tool Search	工具很多时，先搜索并加载相关工具

27. 最推荐的框架

可以把 Function Calling 理解成一个 **AI 应用的控制平面**：



真正的工程能力体现在：

- 1. 工具怎么设计；
- 2. schema 怎么约束；
- 3. 哪些工具暴露给模型；
- 4. 哪些动作必须人工确认；

5. 工具结果如何返回；
6. 错误如何降级；
7. 多工具调用如何编排；
8. 如何记录日志和审计；
9. 如何控制成本和上下文；
10. 如何把 Function Calling 接入 RAG、数据库、工作流和业务系统。

这也是从“会用大模型”走向“会构建 Agent 系统”的关键一步。